

Multi-Client Standards Concierge

An AI assistant that answers a developer's plain-English question about engineering standards and routes it to the correct client -> team -> topic, answering with *that client's* rules only. Built with Cognizant's neuro-san multi-agent framework.

A services company runs many client engagements, each with its own coding standards, deploy approvals, security & licensing rules, and approved tools. Developers rotate between clients and mix up the rules. Ask *"Can I use a GPL library?"* and the concierge figures out which client you're on, which team owns your repo, and which topic your question falls under - then answers for that client. An identity layer limits each developer to the client they are authorized for.

All data is synthetic. No real, proprietary, or personally identifiable data is used.

Prerequisites

- Python 3.13
 - An Anthropic API key (the agents use Claude claude-sonnet)
-

Setup

1. Clone the repository

```
git clone <repo-url>
cd Multi-Client-Standards-Concierge
```

2. Create and activate a virtual environment

```
python -m venv .venv
source .venv/bin/activate      # macOS / Linux
# .venv\Scripts\activate      # Windows
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Set your Anthropic API key

Create a .env file in the project root (the backend loads it automatically via python-dotenv):

```
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env
```

Running the App

Run every command from the project root (the folder that contains backend/ and frontend/) so the HOCON include paths resolve.

Launch the Streamlit web interface

```
streamlit run frontend/run.py
```

Open the URL printed in the terminal (usually `http://localhost:8501`) and chat.

Example prompts (run them in order in one session):

- Hi, I'm amirthap. Can I use a GPL library? -> Banking: prohibited
- How many approvers do I need to deploy to production for Team_B? -> remembered client, no re-typing
- Show me the Automotive client's security standards -> not authorized
- Can I use Chrome? -> not on the approved list -> escalate to the architect

Run the backend directly (headless / scripting)

```
python backend/run.py
```

This invokes the concierge with a built-in test prompt and prints the response.

Project Structure

```
Multi-Client-Standards-Concierge/
|-- backend/
|   |-- run.py                      # Backend entry point; exposes invoke_agent()
|   |-- registries/
|   |   |-- manifest.hocon          # Active agent registry
|   |   |-- standards_concierge.hocon # The three-level agent network
|   |   |-- aaosa.hocon             # AAOSA substitution variables
|   |   |-- llm_config.hocon        # Shared LLM configuration (Anthropic claude-sonnet)
|   |-- coded_tools/
|   |   |-- standards_concierge/
|   |   |   |-- employee_directory.py # Identity / access-control layer
|   |   |   |-- client_resolver.py    # Repo/project -> client (sly_data)
|   |   |   |-- scoped_standards_retriever.py # Reads ONE doc; enforces isolation + access
|   |   |   |-- grounding_checker.py   # Flags claims not supported by the source
|   |   |   |-- constants.py          # Clients / teams / topics
|   |   |   |-- data/                 # 64 synthetic standards docs + lookup tables
|-- frontend/
|   |-- run.py                      # Streamlit chat UI (threads chat_context + sly_data)
|-- requirements.txt
|-- architecture.md
|-- summary.md
|-- README.md
```

How It Works

A three-level hierarchical AAOSA network plus an identity layer:

Agent / Tool	Role
Standards_Concierge	Front desk - talks to the developer; establishes identity & client
Client_Router (L1)	Routes to the correct client engagement
Client_<X> (L2)	Per-client team router (team depth varies per client)
<X>_Team_<Y> (L3)	Topic specialist - picks coding/security/deploy/tools and retrieves
EmployeeDirectory	CodedTool - identity -> authorized client + team

ClientResolver	CodedTool - repo/project -> client, remembered in sly_data
ScopedStandardsRetriever	CodedTool - reads exactly one doc; blocks cross-client access
GroundingChecker	CodedTool - verifies every claim against the retrieved text

See architecture.md and summary.md for details.

Dependencies

Package	Version	Purpose
neuro-san	0.6.71	Multi-agent orchestration framework
streamlit	1.56.0	Web chat interface
python-dotenv	1.0.1	.env file support
aiofiles	>=24.1.0	Async file I/O
langchain-mcp-adapters	>=0.1.7,<1.0	MCP tool adapters for LangChain
langchain-anthropic	>=1.0.0,<2.0	LangChain Anthropic integration

Troubleshooting

ANTHROPIC_API_KEY not found - Ensure your .env file is in the project root, or export the variable before running.

HOCON include / file-not-found - Run from the project root so include "backend/registries/..." paths resolve.

Port already in use - streamlit run frontend/run.py --server.port 8502